

KNMP VisitHub

Design and Development of a Web-Based Booking and Visit Planning System for the Kwame Nkrumah Memorial Park

Prepared by: Bernard Annor **Date:** August 2026 **Version:** 1.0 (final, incorporating nine enhancement rounds)

1. Project Title

KNMP VisitHub — a web-based booking and visit-planning system for the Kwame Nkrumah Memorial Park, Accra, Ghana.

2. Problem Statement

The Kwame Nkrumah Memorial Park (KNMP) is one of Accra's most visited heritage sites, yet visitors planning a trip have no single reliable place to find out what the park offers, what entry costs for their visitor category, or how to reserve a date. At present, a prospective visitor has to piece the information together from word of mouth, informal guides, or a physical visit to the park before a travel date can be fixed. The situation is worse for groups and schools, who often arrive on the same day without notice, and for park staff, who have no way of knowing in advance how many people will show up.

On the management side, the park has no digital record of reservations, no projection of daily attendance, and no mechanism for checking arriving visitors against a booking at the gate. This means staffing cannot be planned around expected crowds, there is no record of who booked what, and busy days simply have to be dealt with as they come. In short, the visitor's planning experience is scattered and uncertain, while the park operates without any visibility into its own demand.

KNMP VisitHub addresses both sides of this problem with one platform: visitors get a public explorer, a proper booking flow with automatic cost calculation and confirmation tickets, and a personal itinerary tool, while administrators and gate staff get a dashboard that covers attractions, fees, bookings, check-ins, users, and visitor analytics.

3. Aim and Objectives

Aim. To design and develop a web-based booking and visit-planning system that lets visitors conveniently plan and manage their visits to the Kwame Nkrumah Memorial Park, while giving administrators practical tools for managing attractions and visitor bookings.

Objectives. The project set out to achieve eleven objectives, all of which were realised and verified during development:

#	Objective	Status
1	Analyse the requirements of visitors and administrators of a tourism attraction booking system	Achieved
2	Design a database for managing users, attractions, bookings, and visit plans	Achieved
3	Develop a secure user authentication mechanism (Manus OAuth)	Achieved
4	Allow visitors to browse, search, and filter available KNMP attractions	Achieved
5	Enable visitors to make and manage visit bookings	Achieved
6	Automatically calculate booking costs based on visitor categories and quantities	Achieved
7	Generate a unique booking reference for each successful booking	Achieved
8	Enable visitors to create and manage a personal visit itinerary	Achieved
9	Provide administrators with a dashboard for managing attractions and bookings	Achieved
10	Test and deploy the completed application	Achieved
11	Identify and document technical debt introduced during development	Achieved

4. Stakeholders

Stakeholder	Role and Interest
Park management (project owner)	Commissioned the system; interested in visitor demand data, revenue visibility, and operational control. Holds ultimate administrative authority.
Park administrators	Day-to-day content and operations staff. Manage attractions, fee categories, bookings, user accounts, park settings, and analytics.
Gate staff	Operational staff at park entrances. Validate tickets quickly, ideally by scanning the QR code on a visitor's PDF ticket, and record arrivals.
Visitors (general public)	The primary users. Need accurate attraction and price information, a dependable booking process, confirmation tickets, and an easy way to plan the day.
Schools and group organisers	A special visitor segment. Organise large parties and benefit from the group and school booking packages with bulk pricing.
Tour guides	Deliver the guided tours booked through the events calendar and benefit from advance visibility of registered participants.

5. Requirements Analysis

Requirements were gathered from the project brief and refined through two actor profiles, one for each primary user. The analysis followed three steps: identify what each actor needs to do, decide what the system must enforce on their behalf, and decide what must never happen (failure modes).

Visitor requirements. A visitor must be able to register and sign in without setting up a new password, since password management is a known source of friction and a security burden. They must be able to browse attractions with pictures and descriptions, search by keyword, and filter by category and location because a park has too many attractions for anyone to skim through a flat list. The booking itself must feel trustworthy: the visitor picks a date (or a multi-day range), chooses how many people are in each fee category, sees the cost update live as they change quantities, and receives a unique reference and a PDF ticket by email so the booking can be shown at the gate. An itinerary tool lets a visitor plan the order of the day and share the plan with travel companions.

Administrator requirements. An administrator must be able to keep attraction and fee content current, see every booking, change booking statuses with automatic visitor notification, manage user accounts and roles, configure the park's daily capacity so the booking form can warn about crowded days, review analytics for planning, and rely on an audit log so sensitive actions (role changes, account changes, content edits) are accountable.

Non-functional requirements. Performance targets were set so the system works at the gate, where seconds matter: pages render within two seconds, the check-in lookup within three. Reliability meant that an email outage must never block a booking. Security meant no stored passwords, role-gated administration, and visitor data scoped to its owner. Usability meant a responsive design that works on a phone in a queue as well as on a desktop in an office.

6. Software Requirements Specification (SRS)

The full SRS is provided as a separate document (SRS.pdf); this section summarises it.

Functional requirements. Eight requirement groups (FR-1 to FR-8) were specified. FR-1 covers the attractions explorer with search and live-counting category and location filters. FR-2 covers the fee display and live cost calculation. FR-3 covers bookings: a unique KNMP-XXXXXX reference per booking, single-day and multi-day date ranges, guided-tour time slots, capacity warnings on the booking form, PDF tickets with QR codes, and automatic ticket email on booking. FR-4 covers personal itineraries with planned times and

notes, PDF export, and shareable links. FR-5 covers authentication (Manus OAuth only), blocked-user rejection, and admin user management with the rule that an administrator cannot demote themselves. FR-6 covers the administrative side: attraction and category management, booking status updates that email the visitor, capacity and threshold configuration, gate QR check-in with manual reference fallback and undo, and checked-in indicators with filters. FR-7 covers analytics: KPIs, a category breakdown with CSV export, monthly revenue trends, and a 14-day visitor forecast against the configured capacity line. FR-8 covers the immutable audit log for role changes, account status changes, content edits, status changes, gate actions, and user creation.

Non-functional requirements. Six were specified (NFR-1 to NFR-6): performance (2s render, 10s analytics queries, 3s gate lookup), security (no stored passwords, role gating, owner-scoped data, secrets outside the codebase), reliability (email failure never blocks operations), usability (responsive, consistent states), maintainability (fully typed codebase with a shared client–server contract and automated tests), and portability (browser-based, container-compatible deployment).

Use cases. Ten use cases (UC-1 to UC-10) cover the visitor flows (browse, book, manage bookings, build an itinerary) and the staff flows (manage content, process bookings, gate check-in, analytics, user management, settings).

Acceptance criteria. The system is accepted when all eleven objectives are demonstrated, every functional requirement is verifiable in the running application, the automated test suite passes, and the deployment serves the full visitor and administrator workflows without intervention.

7. Software Effort Estimation

The project was built under an accelerated 48-hour constraint for the core platform, then extended through nine enhancement rounds. Because the actual effort is known and recorded, the estimate below is anchored in the recorded work rather than a purely predictive model.

Work package	Recorded effort	Equivalent person-days
Requirements analysis and project documentation	~6 hours	0.75
Core platform build (Round 0, 48 hours)	48 hours	6
Enhancement rounds 1–7 (analytics, users, audit, export, email, QR, check-in)	~42 hours	5.25
Enhancement round 8 (park map, group packages, itinerary polish)	~8 hours	1
Enhancement round 9 (events calendar and tour registration)	~7 hours	0.9
Production fix and verification (crypto compatibility, redeployment)	~2 hours	0.25
Documentation set (SRS, testing report, technical debt plan, user manual, consolidated document)	~12 hours	1.5
Total recorded effort	~125 hours	~15.6 person-days

If the same system were estimated from scratch using function points, the eleven delivered use cases, thirteen database tables, nine public-facing pages, and nine administrative pages would fall in the "moderate" complexity band, giving a comparable estimate of roughly 12 to 18 person-days for a small team with a full test suite. The recorded figure of about 15.6 person-days therefore sits comfortably inside the predicted band, which also validates the estimation approach.

8. System Analysis

Process analysis. The visitor process begins before arrival: a visitor explores attractions, builds an itinerary, then books a date. The booking process verifies that the party does not push the day over capacity, calculates the cost from category rates, issues a unique reference, produces a PDF ticket with a QR code, and emails it. At the gate, staff scan the QR code or enter the reference manually, the system checks the booking is not cancelled and not already checked in, and records arrival with an undo path for mistakes. On the management side, every booking can be confirmed or cancelled with automatic notification, content changes are audited, and analytics turn booking history into planning data.

Data analysis. The data model separates four concerns: identity (users with OAuth linkage, role, and active flag), content (attractions with coordinates, opening hours, and categories; fee categories with group package settings; guided-tour slots; published events), transactions (bookings with line items and slot reservations, itinerary items and

share codes, event registrations), and operations (audit events and site settings). Monetary values are stored as integer pesewas throughout, deliberately avoiding floating-point currency errors. All business timestamps are stored in UTC.

Feasibility. Technically, every requirement maps to proven, mainstream technologies, and the serverless database and email provider remove infrastructure management from the project's scope. Economically, the main costs are the transactional email service and hosting, both of which scale with actual usage. Operationally, the system fits existing staff roles: gate staff need only a network-connected device with camera access.

9. System Design

Architecture. KNMP VisitHub is a modern full-stack TypeScript application with a three-tier structure. The frontend is React 19 styled with Tailwind CSS 4 and shadcn/ui components, routed with wouter, and served as a single-page application. The backend is an Express 4 server exposing a strongly-typed API through tRPC v11, with zod input validation and superjson serialization, and all RPC traffic funnelled through a single gateway at `/api/trpc`. The data tier is a TiDB Serverless cluster (MySQL-compatible), accessed through Drizzle ORM with prepared statements. Email is delivered through the Resend transactional API, and PDFs are generated server-side with pdfkit, with QR codes embedded via the qrcode library and decoded client-side with jsqr on the gate page.

React 19 + Tailwind 4 (client)

public pages: Home, Explore (list + park map), Events, Book,
My Bookings, Itinerary, share links

admin pages: DashboardLayout sidebar, nine work areas

| tRPC over `/api/trpc` (superjson)



Express 4 + tRPC server

routers (auth, attractions, bookings, events, analytics...)

db helpers, email delivery, ticket and itinerary PDF generation

| mysql2 prepared statements



TiDB Serverless (MySQL-compatible)

13 tables across identity, content, transaction, and operations domains

Authentication and security design. Sign-in is delegated entirely to Manus OAuth, so the server stores no passwords. Session cookies are validated on every protected request, inactive users are rejected before any procedure logic runs, and an `adminProcedure` middleware gates all administrative endpoints. Visitor data is owner-scoped: a visitor can

only see their own bookings and itinerary. Every destructive administrative action writes an immutable audit event identifying the actor and the target.

Database design. The schema comprises thirteen tables. `users` holds accounts linked to OAuth identities with role and active flags. `attractions` holds park attractions with descriptions, images, locations, opening hours, and map coordinates. `visitor_categories` holds fee categories in pesewas, including group package settings (minimum group size and discount percentage). `bookings` holds one row per booking with the unique reference, date range, status, and check-in timestamp; `booking_items` holds per-category line items; `tour_slots` and `booking_slots` manage guided-tour time selection. `itinerary_items` and `itinerary_shares` support the personal planner; `audit_events` records administrative actions; `site_settings` holds the configurable capacity and warning threshold; `events` and `event_registrations` support the public events calendar and tour registrations.

10. Implementation

The stack was implemented end to end in TypeScript, which lets the client and server share one type contract through tRPC — a change to a procedure's input or output is immediately reflected at the call site on both sides. The frontend pages were built on shadcn/ui primitives for a consistent design system, with the explorer offering both a card list and an interactive Google Map view of the park, and the events page offering a month calendar with day chips and an upcoming-events list. On the server, each business operation lives in a typed procedure with zod-validated input; database work goes through shared helpers in a single `db` module so queries can be tested and reused. PDF tickets are assembled with pdfkit, including the visitor's details, line items, any group discount, tour slots, and an embedded QR code encoding `KNMP-TICKET:<reference>`; itinerary PDFs are generated in the same module. Email flows (booking confirmation, status changes, event registration confirmations) are constructed against the Resend API in a dedicated module and deliberately designed so that an email failure is logged but never prevents the booking operation itself.

The feature set grew through nine rounds. Round 0 delivered the core platform. Rounds 1 to 7 added analytics, user management, audit logging, itinerary export and sharing, email confirmations, QR tickets, capacity warnings, and gate check-in. Round 8 added the interactive park map and group and school booking packages with automatic bulk pricing. Round 9 added the events calendar, through which administrators publish special programs and guided tours and visitors register directly for tours.

Two implementation episodes are worth recording because they shaped the final build. During Round 4, the monthly-trends and category-breakdown analytics queries failed in

production with a generic database error even though the same SQL worked in direct database sessions. Incremental reproduction traced the fault to TiDB's prepared-statement path, which rejects SELECT-list aliases referenced in `ORDER BY`; the fix was to use positional ordering, and three live-database regression tests now guard against recurrence. In Round 9, a Tailwind grid utility failed to render a seven-column calendar on the live build, which was resolved with explicit inline grid styles after the framework-level behaviour could not be relied upon. Finally, after the Round 9 deployment, the live site returned server errors on every route because the reference generator used a browser-only global `crypto` API unavailable in the server container; the fix falls back to Node's `node:crypto` module, and the site has been serving correctly since.

11. Testing

Testing followed a layered strategy built on Vitest 2.1. Business logic and procedure permissions were tested as unit and integration tests with mocked database and email modules; the email tests use a mocked HTTP call against the Resend API so no real messages were sent during automated runs, and a separate test confirms the API key authenticates against the live provider. Where database behaviour was provider-specific (the prepared-statement issue above), queries were tested against the real TiDB development database. Type checking ran over the whole codebase with strict TypeScript settings. Rendered pages were verified visually through dev-server screenshots of the public pages and all nine admin pages, and the critical flows were walked through manually in authenticated browser sessions.

At the final checkpoint the suite contains **87 passing test cases across eight files**, covering the booking lifecycle and cost calculation, unique reference generation, multi-day bookings, tour slots, explorer facets, itinerary export and sharing, user management permissions, audit events, the analytics date-range regression suite, email triggering and failure resilience, status-change emails, capacity checking, site settings, gate check-in with undo, and the events calendar with tour registration. Permission tests deliberately exercise negative cases — non-admins blocked from administrative endpoints, owners blocked from demoting themselves, cancelled bookings rejected at check-in.

Test file	Cases	Focus
auth.logout.test.ts	1	Session logout invalidation
booking.test.ts	46	Booking lifecycle, cost calc, references, multi-day, slots, facets, itinerary, permissions, audit, analytics regression
create-email.test.ts	13	Confirmation email trigger, fallback address, provider failure resilience, status emails, capacity check
email.test.ts	1	Resend API key validation
enhancements.test.ts	11	Email helper behaviour, CSV export format and filtering, 14-day forecast computation
round7.test.ts	9	Site settings, gate check-in and undo, cancelled-booking rejection, user creation
round8.test.ts	3+	Group discount calculation, map coordinate bounds, itinerary export filename
events.test.ts	7	Draft filtering, month queries, capacity enforcement, program gating, visitor registrations

One production-grade defect was found and fixed during development (the ORDER BY alias issue described in Section 10), and the known remaining gap is frontend browser automation: the UI is verified through screenshots and manual flows rather than scripted end-to-end tests. Recommended follow-ups are Playwright-based end-to-end coverage of the booking-to-ticket loop and load testing of the analytics queries at higher booking volumes.

12. Technical Debt

Thirteen technical debt items were registered as they arose, each rated by impact and effort to fix. Two have already been retired: the hardcoded capacity constant (resolved in Round 7 by the configurable Settings page) and the TiDB prepared-statement incompatibility (resolved in Round 4 with positional ordering and regression tests). The remainder are sequenced into three remediation waves.

Wave	Items	Rationale
Wave 1 — Reliability	Email retry queue (lost tickets during provider outages); rate limiting on public endpoints	Directly affect visitor experience and the public surface
Wave 2 — Operational fidelity	Per-party check-in rows; pre-aggregated analytics rollups; shared currency formatter; splitting the router file into per-feature modules; offline-tolerant gate client	Improve gate operations and long-term performance
Wave 3 — Hygiene	Slot-level attendance modelling; API versioning; settings-driven static content; a repeatable database seed fixture	Low-urgency improvements to fold into future feature work

Preventive practice going forward: new procedures go into feature-scoped router files, operational values go into site settings rather than constants, new SQL is covered by live-database tests, and email-sending paths are treated as retryable by default. Every checkpoint continues to require a green test run before delivery. The full register with impact and effort ratings is in [Technical_Debt_Plan.pdf](#).

13. Deployment

The application is deployed on Manus hosting with autoscaling serverless instances: the same Express service serves the React build and the `/api/trpc` gateway, and instances scale to zero when idle. The database is a TiDB Serverless cluster in the US East region, and email delivery is provisioned through a Resend API key stored as a deployment secret rather than in the codebase.

Item	Detail
Live application	https://knmpvisithub-ffuld8d6.manus.space
Admin dashboard	https://knmpvisithub-ffuld8d6.manus.space/admin
Source repository	https://github.com/annorb/knmp_visithub
Database	TiDB Serverless (MySQL-compatible), US East
Email	Resend transactional email (secret-managed API key)
Authentication	Manus OAuth (Google sign-in)

The development preview runs on the project's dev server at port 3000, and every saved checkpoint is automatically published to production. A custom domain can be bound through the hosting dashboard, and the source can be exported at any time through the GitHub integration in the project settings.

14. User Manual

The complete step-by-step manual is provided as `User_Manual.pdf`. In brief:

Visitors sign in through the shared sign-in button (Manus OAuth), then explore attractions through the search box and category and location filters, with an optional map view showing each attraction's position in the park. Booking a visit takes four steps: choose a date or date range, optionally reserve guided-tour slots, pick visitor categories and quantities while the total updates live, and confirm with party details — after which the visitor receives a KNMP-XXXXXX reference, a downloadable PDF ticket with QR code, and an automatic email copy. Existing bookings appear under My Bookings, where the ticket can be re-downloaded or a pending booking cancelled. The Itinerary page lets a visitor build a day plan with planned times and notes, reorder items, export it as a PDF, or share it by a public link. For the events calendar, visitors browse published programs and guided tours on the month grid and register directly for tours, receiving a confirmation email.

Administrators manage the park through nine work areas: an Overview of current bookings and revenue; Attractions and Visitor Categories content management (including group package settings); a Bookings page with status updates that email visitors, checked-in indicators and filters; a Gate Check-in page for QR scanning or manual reference entry with undo; an Analytics page with KPIs, a category breakdown with CSV export, monthly revenue trends, and a 14-day visitor forecast; a Settings page for daily capacity and warning threshold; a Users page for roles, activation, and account creation; and an Audit Log.

Troubleshooting. The most common issues have known remedies: a refused sign-in usually means the account was deactivated and needs reactivation by an administrator; a missing ticket email is recovered from My Bookings; a red capacity banner means the day is crowded and another date should be chosen; a failed QR scan at the gate is worked around with the manual reference field; and stale analytics numbers are cleared by refreshing the page.

15. Maintenance Strategy

Maintenance is organised around four layers. **Corrective** maintenance handles defects as they surface: the recorded process is to reproduce the fault, patch the affected procedure or query, add a regression test before delivery, and verify the fix on the live deployment. The two production incidents already handled (the analytics query failure and the server-side crypto compatibility error) followed exactly this path and each left new tests behind.

Preventive maintenance runs continuously: every checkpoint requires a green test suite, TypeScript checks pass across the whole codebase, dependencies are reviewed through the standard update process, and the technical debt register is revisited so that retired items are never quietly re-introduced.

Adaptive maintenance responds to changes in the environment: updates to the hosting platform, database region behaviour, email provider API changes, and browser API changes (as the crypto compatibility incident showed) are absorbed by keeping external-service calls behind single modules — `email.ts`, `db.ts`, `storage.ts` — so that an adapter change stays local.

Perfective maintenance is planned through the remediation waves in the technical debt plan, folded into future feature work rather than performed as isolated clean-ups. Database changes follow a disciplined migration path (Drizzle schema → generated SQL → applied migration → verified query) so that schema evolution never risks production data. Backups of the TiDB cluster are managed by the hosting platform, and monitoring is available through the deployment dashboard's visibility and analytics controls.

16. Future Evolution

The roadmap builds naturally on what is already deployed. In the near term, an **outbox-based email retry queue** would guarantee ticket delivery across provider outages, and **per-IP rate limiting** on public endpoints would protect the park's booking form from abuse. The gate workflow can evolve toward **per-party attendance counting** with a paper-free daily manifest, and the analytics pipeline toward **materialised rollups** so reports stay fast as years of booking history accumulate.

Mid-term opportunities include an **events reminder service** that emails registered participants before an event, **registration QR codes** on confirmation emails for faster tour check-in at the meeting point, a **suggested walking route** that orders itinerary items by their position on the park map, and a dedicated **Groups & Schools page** documenting the bulk booking process for organisers. An **announcements banner** controlled from admin Settings would let the park publish public notices such as holiday hours or special commemorations.

Longer-term, the platform could support **online payment** for pre-paid entry, **season-ticket and membership** products, **multi-language support** for international visitors, and **integration with the park's existing visitor management** systems. Each of these builds directly on the booking, events, and settings infrastructure already in place.

17. Limitations

It is important to be clear about what the current system does not do. **Email delivery is fire-and-forget**: when the provider is unavailable, ticket emails are silently dropped (visitors can still download tickets from My Bookings). **Check-in is recorded per booking**, not per party member, so gate staff cannot track partial arrivals within a group. **The capacity forecast spreads multi-day stays evenly** across days rather than modelling where guests will actually be each day. **There is no offline tolerance at the gate**: QR scanning requires a network connection, and poor lighting or distance can force a fallback to manual entry. **Public endpoints are not rate limited**, which leaves search and capacity lookup exposed to heavy traffic. **Frontend behaviour is not covered by automated browser tests**, so complex UI interactions rely on screenshots and manual verification. **Analytics read raw rows to aggregate**, which will need rollups as history grows. Finally, **attendance and payment** are not modelled: the system confirms reservations but does not process money or reconcile tickets against actual gate revenue.

18. Conclusion

KNMP VisitHub set out to give visitors of the Kwame Nkrumah Memorial Park a single dependable place to explore the park, understand the fees, book a date, and plan the day — and to give the park's staff the operational tools that go with it. By the close of the ninth enhancement round, all eleven project objectives have been achieved and verified: the attractions explorer covers all eight attractions with search, facets, and a live park map; the booking system issues unique references, calculates costs live, supports multi-day visits and guided-tour slots, and warns about crowded days; tickets carry scannable QR codes and are emailed automatically; itineraries can be planned, exported, and shared; group and school packages apply bulk pricing automatically; the events calendar lets administrators publish programs and tours that visitors register for directly; and the administrative suite covers content, bookings, gate check-in, analytics, settings, user management, and a full audit trail.

The system was delivered through an iterative, checkpoint-driven process with 87 passing automated tests, a documented technical debt register with a sequenced remediation plan, and a production deployment serving the complete visitor and staff workflows. The honest limitations recorded in this document — fire-and-forget email, per-booking check-in, no offline gate support — are not excuses so much as the map of the next phase of work. The foundation, however, is solid: a typed end-to-end architecture, a disciplined schema and migration practice, and a test suite that has already caught and prevented its share of real defects. KNMP VisitHub is ready to serve the park's visitors from today, and ready to grow with the park tomorrow.

19. References

cozmo. (n.d.). *jsQR: A javascript QR code reading library* [Computer software]. GitHub. <https://github.com/cozmo/jsQR>

Drizzle Team. (n.d.). *Drizzle ORM documentation*. <https://orm.drizzle.team>

Google. (n.d.). *Google Maps JavaScript API documentation*. Google Developers. <https://developers.google.com/maps/documentation/javascript>

Google. (n.d.). *OpenID Connect*. Google Identity. <https://developers.google.com/identity/openid-connect/openid-connect>

Manus. (n.d.). *KNMP VisitHub* [Source code]. GitHub. https://github.com/annorb/knmp_visithub

Meta Platforms. (n.d.). *React documentation*. <https://react.dev>

MySQL AB. (n.d.). *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

pdfkit. (n.d.). *pdfkit: PDF generation library* [Computer software]. <http://pdfkit.org>

PingCAP. (n.d.). *TiDB documentation*. <https://docs.pingcap.com/tidb/stable>

Resend. (n.d.). *Resend documentation*. <https://resend.com/docs>

Soldair. (n.d.). *qrcode: QR code generation library* [Computer software]. GitHub. <https://github.com/soldair/node-qrcode>

Tailwind Labs. (n.d.). *Tailwind CSS documentation*. <https://tailwindcss.com>

tRPC. (n.d.). *tRPC documentation*. <https://trpc.io>

Visit Ghana. (n.d.). *Kwame Nkrumah Memorial Park*. Ghana Tourism Authority. <https://www.visitghana.com>

Vitest Authors. (n.d.). *Vitest documentation*. <https://vitest.dev>

Project source code: https://github.com/annorb/knmp_visithub · Live application: <https://knmpvisithub-ffuld8d6.manus.space>